

Documentação de Componentes do Site — Programa Bem Estar (Novartis)

Guia dos componentes visuais e de código já existentes no site, como eles se repetem entre páginas, e onde fica o conteúdo de cada um — referência para recriação do site no WordPress.
Outubro de 2026

1. Visão geral do projeto

O site atual é estático: HTML, CSS e JavaScript puro, sem framework e sem build (não usa React, Vue, Webpack etc.). Todo o conteúdo dinâmico (banners, cards de campanha, grids de condições) é montado por pequenos scripts JavaScript que preenchem a página depois que ela carrega.

Estrutura de pastas

Pasta	O que tem dentro
/pages	Uma página HTML por URL (ex: dermatologia.html, artigo.html)
/partials	Pedaços de HTML reaproveitados em toda página: cabeçalho e rodapé
/css/global	Reset, variáveis de cor/fonte, cabeçalho e rodapé
/css/components	Componentes visuais reutilizáveis entre páginas (botão, banner, card...)
/css/[nome-da-página].css	Estilos exclusivos daquela página
/js	Scripts — alguns genéricos (reutilizados em todo site), outros exclusivos de uma página
/assets	Imagens e ícones

Como uma página é montada

Toda página segue o mesmo esqueleto:

```
<body>
  <div data-include="header"></div>   <- cabeçalho injetado
  <main>
    ... conteúdo exclusivo da página ...
  </main>
  <div data-include="footer"></div>   <- rodapé injetado
</body>
```

O script `js/include.js` busca `partials/header.html` e `partials/footer.html` e insere no lugar desses divs. Cabeçalho e rodapé existem em um único arquivo cada, e qualquer alteração reflete em todas as páginas automaticamente.

No WordPress: isso vira o `header.php` e `footer.php` do tema (ou um *Template Part / Reusable Block*, se construído em blocos).

2. Componentes reutilizáveis (css/components)

Cada componente abaixo é um conjunto de classes CSS (às vezes com um JS que o alimenta) usado em várias páginas diferentes. Alterar o CSS do componente muda a aparência em todo lugar onde ele é usado.

button.css — usado em todas as páginas

O botão padrão do site (`.btn`) e suas variações de cor: `.btn--navy`, `.btn--yellow`, `.btn--outline`, `.btn--white`. Usado tanto em links (`<a>`) quanto em `<button>`.

page-banner.css + js/page-banner.js — dados em JS

Banner de topo (foto + gradiente azul + título + texto) usado nas páginas de especialidade (Cardiologia, Dermatologia, Hematologia, Nefrologia, Neurologia, Oncologia, Pneumologia, Reumatologia) e em páginas institucionais (Campanhas, Medicamentos, FAQ, Especialidades, Fale Conosco).

Na página só existe um "espaço reservado" vazio:

```
<div data-page-banner="dermatologia"></div>
```

O conteúdo (imagem, texto alternativo, título e descrição) vem de um objeto central dentro de `page-banner.js`:

```
PAGE_BANNERS = {
  dermatologia: {
    image: "specialty-dermatologia.png",
    title: "Homem sorridente em área externa",
    alt: "Homem de cabelo afro sorrindo...",
    heading: "Dermatologia",
    description: "Conheça a especialidade médica..."
  },
  ...
}
```

Para criar uma especialidade nova, basta adicionar uma entrada nesse objeto — nenhum HTML precisa ser escrito à mão.

condition-hero.css

Banner parecido visualmente com o anterior, mas usado nas páginas de doença individual (ex: AME, Furúnculo, Infarto, artigo de blog, Área Médica/Parceiros). Diferença importante: aqui o conteúdo é escrito direto no HTML de cada página (não vem de um objeto JS central), reaproveitando as classes `condition-hero` / `condition-hero__image` / `condition-hero__content`.

campaign-card.css + js/campaigns.js — dados em JS

Cartão de campanha (imagem + categoria + título + descrição). Aparece na Home, em `campanhas.html` e na seção "Campanhas:" de cada página de especialidade. Também desenha sozinho o estado "sem campanhas no momento" quando a especialidade não tem nenhuma cadastrada. Todo o conteúdo vem do objeto `CAMPAIGNS` em `campaigns.js`.

related-card.css / related-article-card.css + js correspondentes — dados em JS

Cards de "conteúdo relacionado" / artigos de blog, mostrados ao final das páginas de doença e no blog. Conteúdo vem dos objetos `RELATED` e `RELATED_ARTICLES` nos respectivos arquivos JS.

search-popup.css / search-suggestions.css — usado em todas as páginas

Sistema de busca do site: popup modal (abre ao clicar na lupa do cabeçalho), sugestões de autocomplete enquanto digita, e a página de resultados (`busca.html`). O índice de busca é a lista estática `SEARCH_INDEX` em `search-index.js`.

share-modal.css

Modal de "compartilhar" usado em artigos de blog.

accessibility-widget.css — usado em todas as páginas

Widget flutuante de acessibilidade (ícones no canto da tela), presente em todas as páginas.

utilities.css

Classes utilitárias pequenas e genéricas reaproveitadas em várias páginas: `.tag` (etiqueta) e `.section-title` (título de seção padrão).

3. Padrão repetido nas páginas de especialidade

As 8 páginas de especialidade (Cardiologia, Dermatologia, Hematologia, Nefrologia, Neurologia, Oncologia, Pneumologia, Reumatologia) seguem exatamente a mesma estrutura, só trocando texto/imagens/links:

1. Breadcrumb (Home > Especialidades > Nome) — trunca com reticências o item do meio quando o texto não cabe na tela, automaticamente.
2. Banner — ver `page-banner.js` acima.
3. Introdução — título (`h2`) + dois parágrafos de texto.
4. Bloco "Condições:" — lista de doenças daquela especialidade com ícone e link. Vem do objeto `SPECIALTY_CONDITIONS` em `include.js`, usando `<div data-condition-grid="cardiologia"></div>`.
5. Bloco "Campanhas:" — ver `campaign-card.css` acima, usando `<div data-specialty-campaigns="cardiologia"></div>`.
6. Referências — lista numerada de fontes.

Como as 8 páginas são idênticas em estrutura, no WordPress elas viram um único template ("Página de Especialidade") que recebe campos diferentes por página, em vez de 8 arquivos separados como hoje.

4. Onde fica o conteúdo de cada coisa (JavaScript)

Arquivo	O que monta	Onde está o conteúdo
<code>include.js</code>	Injeta header/footer; monta grids de "Condições:" nas páginas de especialidade e no menu mobile	Objeto <code>SPECIALTY_CONDITIONS</code> (no topo do próprio arquivo)
<code>page-banner.js</code>	Banner de topo das páginas de especialidade/institucionais	Objeto <code>PAGE_BANNERS</code>
<code>campaigns.js</code>	Cards de campanha (Home, Campanhas, seções "Campanhas:")	Objeto <code>CAMPAIGNS</code>
<code>related-articles.js</code>	Cards de artigos relacionados (blog)	Objeto <code>RELATED_ARTICLES</code>
<code>related-cards.js</code>	Cards de "conteúdo relacionado" (páginas de doença)	Objeto <code>RELATED</code>
<code>specialty-grid.js</code>	Grid de condições na página Especialidades (hub)	Reaproveita <code>SPECIALTY_CONDITIONS</code>
<code>specialties-carousel.js</code>	Carrossel de especialidades na Home	Reaproveita <code>SPECIALTY_CONDITIONS</code>
<code>search-index.js</code>	Lista de tudo que a busca encontra	Lista <code>SEARCH_INDEX</code>
<code>[nome-da-página].js</code> (ex: <code>ame.js</code>)	Comportamento exclusivo daquela página (abrir/fechar accordions, trocar abas etc.)	Não tem "dados" — é só interação

Esses objetos (`PAGE_BANNERS`, `CAMPAIGNS`, `SPECIALTY_CONDITIONS` etc.) são, na prática, o "banco de dados" do site hoje. É neles que qualquer pessoa consegue achar e editar o texto de um card ou banner, seguindo o padrão das entradas já existentes.

5. Responsividade (mobile)

Largura	Uso
até 1180px	Menu vira hamburguer, dropdowns do cabeçalho viram lista vertical
até 900px	Grids de 2+ colunas colapsam (ex: condições, cards relacionados)
até 720px / 600px	Banners mudam de layout horizontal para um formato vertical com gradiente por cima da foto; paddings laterais são reduzidos

O banner mobile (gradiente azul de baixo pra cima + recorte de imagem específico) foi padronizado em todas as páginas de especialidade — cada página tem seus próprios valores de recorte de imagem (posição/zoom), pois cada foto tem um enquadramento diferente.

6. Recomendações para a migração ao WordPress

- Cada arquivo em css/components vira um bloco reutilizável (Gutenberg nativo, ACF Flexible Content, ou blocos de um page builder). O nome do arquivo já indica o nome do bloco.
- Os objetos JS de dados (PAGE_BANNERS, CAMPAIGNS, SPECIALTY_CONDITIONS, RELATED_ARTICLES etc.) viram Custom Post Types ou Campos ACF, para edição pelo painel do WP sem precisar mexer em código.
- As 8 páginas de especialidade viram um único template com campos variáveis (ver seção 3), em vez de páginas/arquivos separados.
- header/footer viram Template Parts do tema.
- A nomenclatura das classes CSS já segue um padrão parecido com BEM (.bloco__elemento--variação), o que facilita mapear 1 para 1 ao recriar os estilos no tema do WordPress.